

TP : Paralléliser un calcul de moyenne avec Numba

Ibrahima Gabar Diop
M1 MBDA, Université Cheikh Hamidou Kane

Juin 2026

1 Objectif

Calculer la moyenne d'une promo de N étudiants, d'abord en séquentiel puis en parallèle avec Numba, mesurer le gain de temps (speedup) et estimer la part parallélisable du calcul avec la loi d'Amdahl.

2 Le calcul

Chaque étudiant a trois notes pondérées par des coefficients.

Matière	Coefficient
Maths (M)	5
Physique (P)	4
Anglais (A)	2

La moyenne d'un étudiant vaut $(M \times 5 + P \times 4 + A \times 2)/11$. On la calcule pour chacun, puis on en fait la moyenne sur toute la promo. Les étudiants sont indépendants : aucune note ne dépend d'une autre, donc la boucle peut être répartie sur plusieurs threads sans fausser le résultat.

3 Méthode

Les données sont générées dans un fichier CSV (notes tirées entre 0 et 20). Le calcul est écrit en deux versions, toutes deux compilées par Numba.

La version séquentielle utilise le décorateur `@njit` et s'exécute sur un seul thread : elle sert de référence. La version parallèle utilise `@njit(parallel=True)` avec `prange` : la boucle est répartie sur les threads et la somme finale est gérée comme une réduction, sans condition de course.

Le coeur de la version parallèle :

```
@njit(parallel=True, cache=True)
def moyenne_promo_par(M, P, A):
    total = 0.0
    n = M.shape[0]
    for i in prange(n):
        total += (M[i] * 5 + P[i] * 4 + A[i] * 2) / 11
    return total / n
```

4 Résultats

Mesure sur 10 000 000 d'étudiants, machine à 4 coeurs physiques (8 threads logiques). Le temps retenu est le meilleur sur 100 exécutions, compilation JIT exclue. Les deux versions donnent exactement le même résultat (moyenne de la promo : 10,0004).

Threads	Temps (ms)	Speedup
Séquentiel	18,28	1,00
1	17,20	1,06
2	9,35	1,96
4	8,66	2,11
8	9,88	1,85

Avec un seul thread, la version parallèle rejoint la séquentielle : le surcoût de **prange** est négligeable. Le speedup est presque idéal à 2 threads (1,96) et atteint son maximum à 4 threads (2,11), soit le nombre de coeurs physiques. À 8 threads il redescend, car les threads supplémentaires sont des hyperthreads qui partagent les mêmes coeurs.

5 Speedup et loi d'Amdahl

Le meilleur speedup mesuré est $S = 2,11$ pour $n = 4$ threads. La loi d'Amdahl relie le speedup à la proportion parallélisable p :

$$S = \frac{1}{(1-p) + \frac{p}{n}} \quad \implies \quad p = \frac{1 - \frac{1}{S}}{1 - \frac{1}{n}}$$

En remplaçant :

$$p = \frac{1 - \frac{1}{2,11}}{1 - \frac{1}{4}} = \frac{0,526}{0,75} \approx 0,70$$

La proportion parallélisable du calcul est donc estimée à environ 70 %.

6 Discussion

Le calcul fait peu d'opérations par étudiant mais lit beaucoup de données : il est en partie limité par la bande passante mémoire. C'est pourquoi le speedup plafonne autour de 2 au lieu d'atteindre le nombre de coeurs, et pourquoi l'hyperthreading n'apporte rien. La tâche est parallèle à 100 % sur le papier, mais limitée en pratique par l'accès mémoire, ce que la loi d'Amdahl traduit par un p inférieur à 1. Les valeurs exactes varient un peu selon la charge de la machine ; l'ordre de grandeur (speedup proche de 2, p entre 60 et 70 %) reste stable.

7 Reproduire

```
pip install -r requirements.txt
python generate_data.py --n 10000000 --out etudiants.csv
python benchmark.py --csv etudiants.csv --repeat 100
```

Code complet : <https://github.com/Gblack98/tp-parallelisation-numba>